

BP_ControllerAI – Descripción limpia de la red neuronal y control del vehículo

1. Función general del Blueprint

El **BP_ControllerAI** es el controlador encargado de dirigir cada vehículo inteligente de la simulación. Su función principal es recoger información del entorno, convertirla en entradas numéricas para una red neuronal, ejecutar esa red y transformar sus salidas en acciones de conducción.

Este blueprint no calcula directamente el fitness del coche. Esa tarea pertenece al **BP_VehicleAI2**. Sin embargo, el **BP_ControllerAI** es el responsable de decidir cómo se mueve el vehículo: cuánto gira, cuánto acelera y cuánto frena.

Para tomar esas decisiones, el controlador utiliza una red neuronal simple formada por:

- Una capa de entrada.
- Una capa oculta.
- Una capa de salida.

La red recibe información como velocidad, velocidad angular, sensores de distancia, estado de semáforos, señales de parada, distancia a la línea de detención y tipo de control de tráfico activo. A partir de esos datos genera dos salidas:

- **Steering**, que controla la dirección del coche.
- **Throttle**, que controla aceleración o frenado.

El controlador también contiene funciones para crear cerebros aleatorios, mutar cerebros existentes, cargar pesos ya guardados y validar que la red neuronal tenga una estructura correcta antes de usarla.

2. Variables principales

Variables relacionadas con el coche controlado

MyControlledCar guarda la referencia al **BP_VehicleAI2** que está siendo controlado por este **BP_ControllerAI**.

TrainingMode indica si el coche está participando en entrenamiento o en test. Esta variable se usa en otros blueprints para decidir cómo evaluar el vehículo.

SimulationWithLoadedCar indica si el coche está usando directamente un modelo cargado, normalmente para diferenciarlo visualmente o conceptualmente de los coches mutados o aleatorios.

Variables de control de movimiento

`Steering` almacena el valor actual de giro aplicado al vehículo. Su rango normal es de `-1.0` a `1.0`.

`Throttle` almacena el valor actual de aceleración o frenado. Si es positivo, se interpreta como aceleración. Si es negativo, se interpreta como freno.

`TargetSteering` guarda el valor objetivo de steering calculado por la red neuronal.

`TargetThrottle` guarda el valor objetivo de throttle calculado por la red neuronal.

`MaxSteeringRate` limita cuánto puede cambiar el steering por segundo. Su valor es `1.0`.

`MaxThrottleRate` limita cuánto puede cambiar el throttle por segundo. Su valor es `1.1`.

Estas dos últimas variables permiten suavizar la conducción, evitando que el coche pase instantáneamente de un giro extremo a otro o de acelerar a frenar de forma brusca.

Variables de sensores y entorno

`MaxVisionDistance` define la distancia máxima de visión de los sensores. Su valor es `3000.0`.

`SensorCount` indica cuántos sensores o rayos se lanzan para detectar el entorno. En esta configuración hay `11` sensores.

`FrontObstacleDistance` guarda la lectura del sensor central, que representa la distancia frontal normalizada al obstáculo.

`DistSensorIzda` guarda la lectura del sensor más a la izquierda.

`DistSensorDcha` guarda la lectura del sensor más a la derecha.

`HorizontalDistDiff` representa la diferencia lateral entre sensor derecho e izquierdo.

`RoadWidth` representa una estimación de la anchura de la carretera, usada en el fitness.

`TemporalSensorValue` es una variable temporal usada durante la lectura de sensores.

Variables de red neuronal

`NumHidden` indica cuántas neuronas tiene la capa oculta. En esta versión se usan `16` neuronas.

`InputsForFrame` contiene las entradas preparadas para la red neuronal en el frame actual.

`Weights_IH` almacena los pesos entre la capa de entrada y la capa oculta.

`Weights_HO` almacena los pesos entre la capa oculta y la capa de salida.

`GuardFireCount` cuenta cuántas veces se ha detectado un problema estructural en la red neuronal. Si el problema se repite demasiadas veces, el coche se mata con la razón `INVALID_BRAIN`.

Variables de tráfico e intersecciones

`ForceStop` indica si algún control de tráfico está obligando al coche a detenerse.

`StopTargetLocation` almacena la posición exacta donde el coche debería detenerse, por ejemplo la línea de STOP, YIELD o semáforo.

`TrafficLightState` representa el estado del semáforo como número:

- `0.0` para rojo.
- `0.5` para ámbar.
- `1.0` para verde.

`bIsTrafficLightActive` indica si el coche está actualmente dentro de una zona afectada por un semáforo.

`StoppedCorrectly` indica si el coche ya ha validado correctamente una parada o ceda.

`SensorEntryDistance` guarda la distancia desde el coche hasta la línea de parada en el momento en que entró en la zona de control.

`TrafficControlType` indica qué tipo de control de tráfico afecta actualmente al coche:

- `0` : semáforo.
- `1` : STOP.
- `2` : YIELD.
- `3` : ninguno.

`IntersectionDecision` guarda la dirección elegida por el coche en una intersección.

`CurrentTrigger` guarda el trigger de intersección actual con el que está interactuando el vehículo.

3. Flujo principal: Event Tick

El `Event Tick` es el flujo que actualiza el control del coche en cada frame.

Primero se comprueba si `MyControlledCar` es válido. Si no existe un coche controlado, el blueprint no continúa.

Si el coche es válido, se llama a `PrepareNetworksInputs`. Esta función recopila todos los datos necesarios del vehículo y del entorno, y devuelve un array llamado `ReadyInputs`.

Después se llama a `ProcessAI`, usando `ReadyInputs` como entrada. Esta función ejecuta la red neuronal y devuelve dos valores:

- `Steering` calculado por la red.
- `Throttle` calculado por la red.

Estos valores se guardan como objetivos en `TargetSteering` y `TargetThrottle`.

Después, el controlador actualiza los valores reales de `Steering` y `Throttle`. En lugar de aplicar directamente las salidas de la red, las aproxima progresivamente a los objetivos usando `MaxSteeringRate`, `MaxThrottleRate` y `DeltaSeconds`.

La idea del suavizado es:

```
Steering = Clamp(Steering + Clamp(TargetSteering - Steering, -MaxSteeringRate * DeltaSeconds, MaxSteeringRate * DeltaSeconds), -1, 1)
```

Y para el throttle:

```
Throttle = Clamp(Throttle + Clamp(TargetThrottle - Throttle, -MaxThrottleRate * DeltaSeconds, MaxThrottleRate * DeltaSeconds), -1, 1)
```

Con esto se evita que el coche haga cambios demasiado bruscos entre frames.

Una vez calculados los valores finales, se aplican al componente de movimiento del vehículo:

1. Se llama a `Set Steering Input` con el valor de `Steering`.
2. Se comprueba si `Throttle >= 0.0`.
3. Si `Throttle` es positivo, se pone el freno a `0.0` y se aplica throttle positivo.
4. Si `Throttle` es negativo, se usa su valor absoluto como freno y se pone el throttle real a `0.0`.

De esta forma, la red neuronal solo necesita producir un único valor de salida para aceleración/freno. Valores positivos aceleran y valores negativos frenan.

4. Eventos auxiliares de color

El controlador tiene dos eventos simples para cambiar el color del coche según el tipo de cerebro que esté usando.

`BestCarColor` llama a `SetColorBlue` sobre `MyControlledCar`. Se usa para identificar visualmente el coche que representa el mejor modelo de la sesión.

`LoadedCarColor` llama a `SetColorGreen` sobre `MyControlledCar`. Se usa para identificar visualmente el coche que usa el modelo cargado desde un archivo guardado.

Estos eventos no afectan al comportamiento de conducción, pero ayudan a interpretar visualmente la simulación.

5. Posesión del coche: Event OnPossess

Cuando el controlador posee un pawn, se ejecuta `Event OnPossess`.

Primero se convierte el pawn poseído a `BP_VehicleAI2`. Si la conversión es válida, se guarda la referencia en `MyControlledCar`.

Después se reinician los valores de control:

- `Steering = 0.0`
- `Throttle = 0.0`
- `TargetSteering = 0.0`
- `TargetThrottle = 0.0`

Finalmente, `TrafficControlType` se pone en `3`, indicando que al inicio no hay ningún control de tráfico activo.

Inicialización y carga de cerebros

6. Inicialización de cerebro aleatorio: InitializeRandomBrain

`InitializeRandomBrain` crea una red neuronal nueva con pesos aleatorios.

Esta función se usa cuando el sistema necesita introducir diversidad en la población o cuando no existe un modelo previo cargado.

Cálculo del número de entradas

Primero se calcula el número total de entradas de la red.

La fórmula es:

```
NumInputs = SensorCount + 2 + 4 + 4 + 1
```

Con la configuración actual:

- `SensorCount = 11`, por los sensores de distancia.
- `2` entradas adicionales para velocidad y velocidad angular.
- `4` entradas para información de tráfico: semáforo activo, estado del semáforo, distancia a la línea de parada y `ForceStop`.
- `4` entradas one-hot para el tipo de control de tráfico: semáforo, STOP, YIELD o ninguno.
- `1` entrada de bias.

En total:

```
11 + 2 + 4 + 4 + 1 = 22 entradas
```

Después se fija `NumHidden` con el valor global del controlador, que actualmente es `16`.

El número de salidas es `2`, correspondientes a steering y throttle.

Creación de pesos Input-Hidden

Después se crea el array `Weights_IH`, que contiene los pesos entre la capa de entrada y la capa oculta.

El número de pesos de esta parte es:

```
NumHidden * NumInputs
```

Con los valores actuales:

```
16 * 22 = 352 pesos
```

Para cada peso, se genera un valor aleatorio entre `-0.5` y `0.5`. Cada valor se añade a `Weights_IH`.

Mientras se generan los pesos, se acumula la magnitud absoluta de cada valor en `TotalMag`. Esto permite calcular después la magnitud media de los cambios o pesos generados.

Creación de pesos Hidden-Output

Después se crea el array `Weights_HO`, que contiene los pesos entre la capa oculta y la capa de salida.

El número de pesos de esta parte es:

```
NumHidden * NumOutputs
```

Con los valores actuales:

```
16 * 2 = 32 pesos
```

Cada peso se genera también de forma aleatoria entre `-0.5` y `0.5` y se añade a `Weights_H0`.

Actualización del EvolutionManager

Cuando termina la inicialización aleatoria, se obtiene el actor `BP_EvolutionManager`.

Después se copian los pesos generados a:

- `BestWeights_InputHidden`
- `BestWeights_HiddenOutput`

Esto se hace como medida de seguridad para que exista un conjunto inicial de pesos aunque no haya un archivo de modelo guardado previamente.

Salidas de depuración

Al final, se calcula:

```
TotalWeights = Length(Weights_IH) + Length(Weights_HO)
```

Con la configuración actual:

```
352 + 32 = 384 pesos
```

La función devuelve:

- `OutNumChanged = TotalWeights`
- `OutAvgMag = TotalMag / TotalWeights`
- `OutNumLarge = TotalWeights`

En un cerebro aleatorio, todos los pesos se consideran generados desde cero, por eso el número de cambios coincide con el número total de pesos.

7. Inicialización de cerebro mutado: InitializeMutatedBrain

`InitializeMutatedBrain` crea una nueva red neuronal a partir de los pesos de un padre.

Recibe tres entradas:

- `ParentWeightsIH`, pesos del padre entre entrada y capa oculta.
- `ParentWeightsHO`, pesos del padre entre capa oculta y salida.
- `MutationRate`, probabilidad de mutar cada peso.

Primero limpia los arrays `Weights_IH` y `Weights_HO` para evitar mezclar pesos antiguos con los nuevos.

Mutación de pesos Input-Hidden

La función recorre todos los pesos de `ParentWeightsIH`.

Para cada peso, genera un número aleatorio entre `0.0` y `1.0`. Si ese número es menor que `MutationRate`, el peso se muta. Si no, se copia tal cual.

Cuando un peso se muta, se incrementa `MutCount`.

Después se decide si la mutación será grande o pequeña:

- Con un `1%` de probabilidad se aplica una mutación grande.
- En el resto de casos se aplica una mutación pequeña.

Una mutación grande genera un `DeltaIH` entre `-0.5` y `0.5`.

Una mutación pequeña genera un `DeltaIH` entre `-0.05` y `0.05`.

El nuevo peso se calcula como:

```
CurrentWeight_IH = PesoPadre + DeltaIH
```

El valor absoluto de `DeltaIH` se añade a `TotalMag`. Si la mutación fue grande, también se incrementa `LargeCount`.

Finalmente, el peso resultante se añade a `Weights_IH`.

Mutación de pesos Hidden-Output

Después se repite el mismo proceso para los pesos `ParentWeightsHO`.

Cada peso puede copiarse sin cambios o mutarse según `MutationRate`.

Si se muta:

- Puede recibir una mutación grande entre `-0.5` y `0.5`.
- O una mutación pequeña entre `-0.05` y `0.05`.

El nuevo peso se añade a `Weights_H0`.

Salidas de depuración

Al final, la función devuelve:

- `OutNumChanged = MutCount`, número total de pesos mutados.
- `OutAvgMag = TotalMag / MutCount`, magnitud media de los cambios aplicados.
- `OutNumLarge = LargeCount`, número de mutaciones grandes.

Si no se ha mutado ningún peso, la división se protege usando `MAX(MutCount, 1)`.

Estas salidas permiten registrar en el coche cuántos cambios recibió su cerebro y con qué intensidad.

8. Carga de cerebro guardado: LoadBrain

`LoadBrain` permite cargar directamente un conjunto de pesos ya existente.

Recibe:

- `LoadWeightsIH`
- `LoadWeightsHO`

Después simplemente copia esos arrays a:

- `Weights_IH`
- `Weights_HO`

Esta función se usa principalmente en modo test o cuando se quiere generar un coche con el mejor modelo guardado.

Lectura del entorno

9. Lectura de sensores: GetSensorsData

`GetSensorsData` calcula las lecturas de distancia del coche usando trazas en forma de abanico.

La función devuelve un array llamado `SensorReadings`.

Configuración de los rayos

Primero se define:

- `NumRayos = SensorCount`
- `VisionAngle = 180.0`

Con `SensorCount = 11`, el coche lanza 11 sensores repartidos en un ángulo de 180 grados delante de él.

Antes de empezar, se limpia el array temporal `TempReadings`.

Ejecución de cada sensor

Para cada índice del bucle, se inicializa `TemporalSensorValue` a `1.0`. Este valor significa que, por defecto, no se ha detectado ningún obstáculo cercano.

Después se comprueba si el pawn controlado es válido.

Si lo es, se lanza un `MultiSphereTraceByChannel` desde una posición elevada sobre el coche hasta un punto calculado a partir de:

- `MaxVisionDistance`
- El vector frontal del coche.
- Una rotación angular dentro del rango de `-90` a `+90` grados.
- Una altura adicional de `120` unidades.

El radio de la esfera usada en el trace es `40.0`. El canal usado es `SensorIA`. El propio coche se ignora para evitar que se detecte a sí mismo.

Tratamiento de impactos

Después del trace, se recorre el array de impactos con un `ForEachLoopWithBreak`.

Si el actor impactado no es un `BP_IntersectionBorders`, se considera un obstáculo normal. En ese caso, `TemporalSensorValue` se calcula como:

`DistanciaImpacto / MaxVisionDistance`

Si no hay impacto, se mantiene el valor `1.0`.

Si el actor impactado sí es un `BP_IntersectionBorders`, se comprueba si ese borde corresponde a la decisión actual del coche:

```
IntersectionDecision == DirectionIntent del borde
```

Y además:

```
CurrentTrigger == Trigger del borde
```

Solo si ambas condiciones se cumplen, el borde se considera relevante para el coche y se guarda la distancia normalizada.

Si el borde de intersección no corresponde a la decisión del coche, se ignora para no penalizar o condicionar una ruta que no debería afectar a esa trayectoria.

Al terminar el procesamiento del impacto, se rompe el bucle de hits y se añade `TemporalSensorValue` a `TempReadings`.

Resultado de los sensores

Cuando terminan todos los rayos, la función devuelve `TempReadings` como `SensorReadings`.

Cada valor del array suele estar entre `0.0` y `1.0`:

- Valores cercanos a `0.0` indican un obstáculo cercano.
 - Valores cercanos a `1.0` indican espacio libre hasta la distancia máxima de visión.
-

10. Preparación de entradas para la red: `PrepareNetworksInputs`

`PrepareNetworksInputs` construye el vector de entrada que recibirá la red neuronal en el frame actual.

Primero limpia `InputsForFrame`.

Después añade los datos en un orden concreto. Este orden debe mantenerse siempre, porque los pesos de la red neuronal dependen de la posición exacta de cada entrada.

Entrada 1: velocidad normalizada

La primera entrada es la velocidad frontal del vehículo normalizada:

```
ForwardSpeed / MaxSpeed
```

El resultado se limita entre `0.0` y `1.0`.

Esta entrada informa a la red de cuánto está avanzando el coche respecto a su velocidad máxima.

Entrada 2: velocidad angular normalizada

La segunda entrada es la velocidad angular en el eje Z.

Se obtiene con `GetPhysicsAngularVelocityInDegrees` del mesh del coche y se toma solo el valor `Z`.

Después se aplica un `Map Range Clamped` de `-90` a `90`, devolviendo un valor entre `-1` y `1`.

Esta entrada informa a la red de si el coche está rotando hacia un lado u otro.

Entradas de sensores

Después se llama a `GetSensorsData`.

La salida `SensorReadings` se guarda en una variable local `SensorsArray`.

Luego se recorre ese array y se añade cada lectura a `InputsForFrame`.

Con `SensorCount = 11`, se añaden 11 entradas.

Después se guardan algunas lecturas específicas en variables globales:

- `FrontObstacleDistance` toma el sensor central, ubicado en el índice `SensorCount / 2`.
- `DistSensorIzda` toma el sensor del índice `0`.
- `DistSensorDcha` toma el sensor del índice `SensorCount - 1`.

Estas variables son usadas posteriormente por el fitness y por otras comprobaciones.

Entradas relacionadas con semáforos y paradas

Después se añaden cuatro entradas de contexto de tráfico.

La primera indica si hay un semáforo activo:

- `1.0` si `bIsTrafficLightActive` es verdadero.
- `0.0` si es falso.

La segunda entrada es `TrafficLightState`, que puede valer:

- `0.0` para rojo.
- `0.5` para ámbar.
- `1.0` para verde.

La tercera entrada es la distancia normalizada hasta `StopTargetLocation`. Esta distancia solo se usa realmente cuando hay un semáforo activo o `ForceStop` está activo. Si no hay control de parada, se añade `1.0`.

La fórmula usada cuando sí hay control de parada es:

```
Clamp(Distance(CarLocation, StopTargetLocation) / Max(SensorEntryDistance, 1.0),  
0.0, 1.0)
```

Esto indica a la red cuánto se ha acercado el coche a la línea de parada desde que entró en la zona.

La cuarta entrada representa `ForceStop`:

- `1.0` si el coche debe detenerse.
- `0.0` si no debe detenerse.

Entradas one-hot del tipo de control

Después se añaden cuatro entradas para representar `TrafficControlType` en formato one-hot.

Este formato permite que la red distinga claramente qué tipo de control está afectando al coche.

Las entradas son:

1. `1.0` si `TrafficControlType == 0`, semáforo; si no, `0.0`.
2. `1.0` si `TrafficControlType == 1`, STOP; si no, `0.0`.
3. `1.0` si `TrafficControlType == 2`, YIELD; si no, `0.0`.
4. `1.0` si `TrafficControlType == 3`, ningún control; si no, `0.0`.

Entrada de bias

Al final se añade una entrada constante de valor `1.0`.

Esta entrada funciona como bias de la red neuronal. Permite que las neuronas puedan activar salidas aunque el resto de entradas sean bajas.

Resultado final

La función devuelve `InputsForFrame` como `ReadyInputs`.

Con la configuración actual, el vector tiene 22 entradas:

- 2 entradas de movimiento.
- 11 entradas de sensores.
- 4 entradas de contexto de parada.
- 4 entradas one-hot de tipo de control.
- 1 entrada de bias.

Procesamiento de la red neuronal

11. Ejecución de la red: ProcessAI

`ProcessAI` recibe como entrada `AllInputs`, que es el vector preparado por `PrepareNetworksInputs`.

Su objetivo es calcular las dos salidas de la red neuronal:

- `Steering`
- `Throttle`

Antes de hacer el cálculo, la función realiza varias comprobaciones de seguridad para evitar errores si los pesos o la estructura de la red no son válidos.

12. Comprobaciones de seguridad

Primero se copia `NumHidden` en una variable local llamada `NumHiddenLayers`.

Después se comprueba que el número de neuronas ocultas sea válido:

```
NumHiddenLayers <= 0 OR NumHiddenLayers > 512
```

Si esta condición se cumple, significa que la red tiene una configuración incorrecta y se activa la salida de seguridad, llamada aquí `GuardExit`.

Después se copian las entradas a la variable local `Inputs`.

La siguiente comprobación revisa que el número de entradas sea exactamente `22`:

```
Length(Inputs) != 22
```

Si no hay 22 entradas, la red no puede procesarse correctamente, porque los pesos esperan un tamaño fijo.

Después se copian los pesos a variables locales:

- `WItH = Weights_IH`
- `WHtO = Weights_HO`

Luego se comprueba el tamaño de `WItH`:

```
Length(WItH) != NumHiddenLayers * Length(Inputs)
```

Con 16 neuronas ocultas y 22 entradas, este array debería tener 352 pesos.

Después se comprueba el tamaño de `WHtO`:

```
Length(WHtO) != NumHiddenLayers * 2
```

Con 16 neuronas ocultas y 2 salidas, este array debería tener 32 pesos.

Finalmente se comprueba que `Weights_IH` no esté vacío.

Si cualquiera de estas comprobaciones falla, se activa `GuardExit`.

13. GuardExit e INVALID_BRAIN

`GuardExit` es una salida de seguridad para evitar que una red mal formada provoque errores de ejecución.

Cuando se activa, se incrementa `GuardFireCount`.

Si `GuardFireCount >= 5`, se considera que el cerebro del coche es inválido de forma persistente. En ese caso:

1. Se llama a `KillCar` sobre `MyControlledCar`.
2. La causa de muerte es `INVALID_BRAIN`.
3. La función devuelve `Steering = 0.0` y `Throttle = 0.0`.

Si `GuardFireCount` todavía es menor que `5`, la función no mata al coche inmediatamente. En su lugar, devuelve los valores actuales de `Steering` y `Throttle`, manteniendo temporalmente el último control conocido.

Esta lógica evita matar al coche por un fallo puntual, pero sí lo elimina si el problema se repite varias veces.

14. Cálculo de la capa oculta

Si todas las comprobaciones son correctas, se limpian los arrays:

- `HiddenLayerValues`
- `FinalOutputs`

Después se calcula la capa oculta.

El sistema ejecuta un bucle desde `0` hasta `NumHiddenLayers - 1`. Cada iteración representa una neurona oculta.

Para cada neurona oculta:

1. `TempSum` se pone en `0.0`.
2. Se recorren todas las entradas.
3. Para cada entrada, se multiplica el valor de entrada por su peso correspondiente.
4. El resultado se suma a `TempSum`.

El índice del peso se calcula como:

```
Length(Inputs) * HiddenIndex + InputIndex
```

Cuando se han procesado todas las entradas de una neurona oculta, se aplica la función de activación:

```
tanh(TempSum)
```

El resultado se añade a `HiddenLayerValues`.

La función `tanh` devuelve valores entre `-1.0` y `1.0`, lo que ayuda a mantener las activaciones acotadas.

Después de calcular todas las neuronas ocultas, se comprueba que el número de valores en `HiddenLayerValues` coincida con `NumHiddenLayers`. Si no coincide, se activa `GuardExit`.

15. Cálculo de la capa de salida

Si la capa oculta se ha calculado correctamente, se calcula la capa de salida.

La red tiene dos salidas, por eso se ejecuta un bucle de índice `0` a `1`:

- Índice `0`: salida de steering.
- Índice `1`: salida de throttle.

Para cada salida:

1. `TempSum` se reinicia a `0.0`.
2. Se recorren todos los valores de la capa oculta.
3. Cada valor oculto se multiplica por su peso correspondiente en `Wht0`.
4. El resultado se suma a `TempSum`.

El índice del peso se calcula como:

```
HiddenIndex + NumHiddenLayers * OutputIndex
```

Cuando se han procesado todas las neuronas ocultas para una salida, se aplica de nuevo:

```
tanh(TempSum)
```

El resultado se añade a `FinalOutputs`.

Al terminar las dos salidas, se reinicia `GuardFireCount` a `0`, porque la red se ha procesado correctamente.

Finalmente, la función devuelve:

- `Steering = FinalOutputs[0]`
- `Throttle = FinalOutputs[1]`

Ambos valores quedan en el rango aproximado de `-1.0` a `1.0` gracias a la función `tanh`.

Resumen general del flujo

16. Funcionamiento completo del `BP_ControllerAI`

El funcionamiento del `BP_ControllerAI` puede resumirse como un ciclo continuo de percepción, decisión y acción.

Primero, cuando el controlador posee un coche, guarda la referencia al `BP_VehicleAI2` y reinicia las variables de conducción. A partir de ese momento, en cada `Tick`, comprueba si el coche existe y empieza el proceso de control.

El controlador recopila información mediante `PrepareNetworksInputs`. Esta función añade al vector de entrada la velocidad, la velocidad angular, las lecturas de sensores, el estado de semáforos, la distancia a la línea de parada, la orden de parada y el tipo de control de tráfico activo.

Después, `ProcessAI` ejecuta la red neuronal usando esos datos. La red tiene 22 entradas, 16 neuronas ocultas y 2 salidas. Las salidas se calculan mediante sumas ponderadas y función de activación `tanh`.

Las dos salidas de la red representan la intención de conducción: cuánto girar y cuánto acelerar o frenar. Esos valores no se aplican de forma completamente instantánea, sino que se suavizan usando límites de cambio por segundo. Así se obtiene una conducción menos brusca.

Finalmente, el controlador aplica el steering al componente de movimiento del coche. Si el throttle calculado es positivo, acelera. Si es negativo, lo convierte en freno.

Además, el blueprint contiene funciones para crear cerebros aleatorios, mutar cerebros existentes y cargar pesos guardados. Estas funciones son usadas por el `BP_EvolutionManager` para generar poblaciones de coches durante el entrenamiento y para probar modelos ya aprendidos durante el test.

En conjunto, este blueprint es el núcleo de decisión del vehículo. Recibe el estado del mundo, lo transforma en entradas numéricas, ejecuta una red neuronal y convierte sus salidas en controles físicos del coche.